YT-DLP Studio

Documentação Técnica

Arquitetura, funcionamento interno e guia de manutenção/extensão

Versão do documento — 03/11/2026

Sumário

1. Visão geral e stack tecnológica

2. Estrutura de arquivos do projeto

3. Fluxo de execução (o que acontece ao abrir o programa)

4. Módulo por módulo

5. Como os comandos do yt-dlp são montados (downloader.py)

6. O mecanismo de recorte de capa (--ppa e escaping)

7. Gerenciamento de dependências (yt-dlp + FFmpeg)

8. Fila de downloads e threading

9. Persistência de configurações

10. Internacionalização (i18n)

11. Guia prático: como adicionar/alterar coisas comuns

12. Build e empacotamento (gerando o .exe)

13. Solução de problemas para desenvolvedores

14. Versão Tauri + TypeScript (pasta tauri/)

1. Visão geral e stack tecnológica

O YT-DLP Studio é um aplicativo desktop escrito em **Python 3**, usando **CustomTkinter** (biblioteca de UI sobre o Tkinter padrão) para a interface gráfica. Ele não reimplementa nenhuma lógica de download: todo o trabalho pesado é feito pelo binário oficial do **yt-dlp** e pelo **FFmpeg**, chamados como processos externos via subprocess. O programa é, em essência, um construtor de linha de comando com interface gráfica, mais um gerenciador de fila/dependências em volta disso.

Peça	Tecnologia	Papel
Interface gráfica	CustomTkinter (Tkinter)	Janelas, abas, formulários
Motor de download	yt-dlp (binário externo)	Extração e download real
Processamento de mídia	FFmpeg (binário externo)	Conversão, corte, capa, mux
Empacotamento	PyInstaller	Gera o .exe standalone
Persistência	JSON (settings.json)	Configurações do usuário

2. Estrutura de arquivos do projeto

```
ytdlp_studio/
|-- build.bat              # gera o .exe (Windows, via PyInstaller)
|-- requirements.txt
|-- assets/
|   |-- icon.ico / icon.png    # icone do app
|   |-- logo.png               # logo horizontal (README/splash)
|   `-- theme_red.json         # tema CustomTkinter preto/branco/vermelho
|-- docs/                  # os dois PDFs (usuario + tecnico)
`-- src/
    |-- main.py            # ponto de entrada, janela principal
    |-- theme.py           # cores compartilhadas (vermelho, cinza, contorno)
    |-- i18n.py            # strings PT/EN
    |-- settings_store.py  # settings.json (ler/gravar)
    |-- dependencies.py    # baixa/verifica yt-dlp e FFmpeg
    |-- downloader.py      # monta comandos yt-dlp + fila (thread)
    |-- tabs.py            # abas Audio e Video
    |-- queue_tab.py       # aba Fila e Log
    |-- settings_dialog.py # janela de Configuracoes
    |-- help_dialog.py     # janela de Ajuda
    `-- widgets.py         # RangeSlider (slider de recorte)
```

3. Fluxo de execução

1. **main.py** cria a classe App(ctk.CTk), que carrega SettingsStore (lê settings.json) e Translator (define o idioma da UI).
2. A barra superior (_build_topbar) e o conteúdo (_build_content) são montados. As três abas (AudioTab, VideoTab, QueueTab) são instanciadas de uma vez e empilhadas na mesma célula de grid — trocar de aba é só chamar .tkraise() na aba escolhida.
3. 300 ms depois da janela abrir, check_dependencies(initial=True) roda em uma *thread* separada, verificando/baixando yt-dlp e FFmpeg sem travar a interface.

4 Um laço _poll_queue_events roda a cada 150 ms via self.after(150, ...), lendo uma queue.Queue onde threads de fundo (dependências, downloads) depositam eventos. Esse é o único ponto onde widgets são atualizados — nunca direto de dentro de uma thread.

5 Quando o usuário clica em Download/Adicionar à fila, um objeto downloader.Job é criado e entregue ao QueueManager, que processa a fila sequencialmente em sua própria thread.

4. Módulo por módulo

main.py — App

Classe principal da janela. Responsabilidades: montar a barra superior (abas + engrenagem + ajuda + tema + idioma), alternar entre as três abas, abrir os diálogos de Configurações/Ajuda, rodar dependências em thread, e o laço _poll_queue_events que traduz eventos da fila em atualizações de UI. Também guarda build_output_template(job), que decide o caminho/nome final do arquivo baixado a partir da pasta configurada e do título/artista do job.

i18n.py

Um dicionário STRINGS = {'pt': {...}, 'en': {...}} com todas as strings da interface por chave. A classe Translator guarda o idioma atual e expõe .t(chave, **kwargs), que busca a string e aplica .format(**kwargs) se houver parâmetros (ex.: tr.t('dep_downloading', size='~80MB')).

settings_store.py

SettingsStore guarda um dicionário com todas as preferências (pastas, opções de rede/filtros/comportamento, últimos valores escolhidos nas abas, geometria da janela). DEFAULTS define os valores padrão — ao carregar, valores salvos são mesclados por cima dos padrões, então adicionar uma nova configuração no futuro nunca quebra um settings.json antigo (a chave nova simplesmente usa o padrão até o usuário mudar). Por padrão o arquivo fica em data/settings.json, bem ao lado do executável (ver seção 7 para os detalhes de como essa pasta é resolvida, com fallback para %APPDATA%).

dependencies.py

Resolve os caminhos de yt-dlp.exe/ffmpeg.exe/ffprobe.exe dentro de uma pasta bin/ ao lado do settings.json. check_ytdlp()/check_ffmpeg() rodam --version para confirmar que o binário existe e funciona. install_ytdlp() baixa o executável mais recente direto do GitHub Releases; install_ffmpeg() baixa o pacote 'essentials' (Windows) e extrai só os três executáveis do .zip. A classe DependencyManager orquestra tudo isso e reporta progresso via um callback status_cb(componente, estado, extra).

downloader.py

O módulo mais importante para manutenção. Contém: os dicionários de mapeamento UI→flag do yt-dlp (ex.: RES_HEIGHT, VCODEC_FILTER, AUDIO_QUALITY_VALUE), a função build_command(job, output_template) que monta a lista de argumentos final, e a classe QueueManager que executa os jobs em uma thread de fundo. Detalhado na seção 5.

tabs.py

DownloadTabBase é a classe-mãe com tudo que Áudio e Vídeo têm em comum: caixa de link, slider de recorte, botões de pasta, Download/Adicionar à fila, status e barra de progresso. AudioTab e VideoTab só implementam build_options_row() (os campos específicos de cada uma) e gather_tab_options() (o dicionário de opções que vira job.options).

queue_tab.py

Interface da aba Fila e Log: DepStatusRow (bolinha colorida + texto de status), QueueRow (um item da fila, com sua própria barra de progresso) e a classe QueueTab que organiza tudo, incluindo o painel de log (append_log).

settings_dialog.py / help_dialog.py

Janelas CTkToplevel independentes. SettingsDialog monta cada seção (Pastas, Rede, Filtros, Comportamento, Arquivos extras) a partir de pequenas classes auxiliares (LabeledEntry, LabeledCheck) que sabem carregar/salvar seu próprio valor de/para o SettingsStore. HelpDialog apenas renderiza o conteúdo estático de HELP_CONTENT (um dicionário pt/en de seções).

widgets.py

RangeSlider: um slider de dois cabos desenhado à mão com tkinter.Canvas (CustomTkinter não tem um componente nativo para isso). Funções auxiliares seconds_to_hms/hms_to_seconds convertem entre segundos e o formato HH:MM:SS mostrado nos campos de texto. Como o Canvas é Tk puro (não CustomTkinter), ele não recebe automaticamente a troca de tema claro/escuro em tempo de execução — por isso existe o método refresh_theme(), chamado explicitamente por App.toggle_theme() para recalcular a cor de fundo do canvas sempre que o tema muda.

theme.py + assets/theme_red.json

A paleta preto/branco/vermelho (#FF0000, igual à identidade visual do yt-dlp) tem duas camadas: assets/theme_red.json é um tema completo do CustomTkinter (mesmo schema do tema 'blue' embutido na biblioteca, só que com os azuis trocados por vermelho), carregado uma vez em main.py via ctk.set_default_color_theme(...) — isso já cobre automaticamente qualquer widget sem cor explícita (entradas, checkboxes, menus, abas). theme.py complementa com constantes Python (PRIMARY_KW, CANCEL_KW, SECONDARY_KW) para os poucos botões que precisam de estilo diferente do padrão — em especial CANCEL_KW, que desenha um botão 'fantasma' (fundo transparente, contorno vermelho) para as ações de cancelar não ficarem visualmente idênticas aos botões vermelhos sólidos de ação primária (Download, Iniciar Fila). Para mudar o tom de vermelho usado em todo o app, basta editar essas duas fontes; não há cor de tema espalhada em outros arquivos.

5. Como os comandos do yt-dlp são montados

Cada download é representado por um objeto downloader.Job: modo ('audio' ou 'video'), URL, título/artista opcionais, e um dicionário options. Esse dicionário é montado em tabs.py → DownloadTabBase._make_jobs() mesclando **todas** as configurações globais (dict(self.app.settings.data)) com as opções específicas daquela aba (formato, resolução, etc.) — por isso, dentro de build_command, a variável s = job.options serve tanto para ler configurações globais quanto opções da aba, sem distinção.

A montagem segue sempre a mesma ordem em build_command():

1. Flags base: caminho do yt-dlp, URL, --newline --no-color --no-warnings --ignore-errors, localização do FFmpeg, e o template de saída (-o).
2. Flags de comportamento geral (restrict-filenames, no-playlist, download-archive, geo-bypass, etc.) — cada uma checando s.get('chave').
3. Flags numéricas de rede/robustez (fragmentos paralelos, tentativas, limite de velocidade, proxy, playlist-items, filtros de data/tamanho/duração).
4. Recorte de trecho: se trim_start/trim_end foram definidos pelo slider, adiciona --download-sections '*INÍCIO-FIM' --force-keyframes-at-cuts.
5. Arquivos extras (metadados, capítulos, descrição, thumbnail).
6. Bloco específico de áudio (-x --audio-format --audio-quality, capa quadrada, metadados via --postprocessor-args) **ou** bloco específico de vídeo (-f com o seletor de formato, contêiner, legendas) — nunca os dois.
7. Por fim, se o usuário preencheu um template avançado em Configurações → Rede, ele substitui o -o padrão.

Exemplo de seletor de formato de vídeo

build_format_selector() traduz Resolução/FPS/Codec em filtros do seletor -f do yt-dlp, sempre com uma alternativa de fallback para nunca falhar por excesso de restrição:

```
# Resolução 1080p + codec H.264 vira:
bestvideo[height<=1080][vcodec~='^avc1']+bestaudio
/best[height<=1080]
```

Ou seja: tenta achar vídeo+áudio separados que batam com os filtros; se não achar, cai para o melhor stream único disponível dentro da resolução pedida. Para adicionar um novo codec, basta acrescentar uma entrada em VCODEC_FILTER (downloader.py) e na lista de opções da UI (tabs.py) — ver seção 11.

Pasta de playlist e status ao vivo (--print)

Antes de cada job de verdade, QueueManager._resolve_playlist_folder() roda um comando rápido e separado — a mesma estratégia usada no script .ps1 original — para descobrir se o link é uma playlist:

```
yt-dlp --get-filename --no-warnings -o "%(playlist_title)s"
       URL --playlist-items 1
```

Se o resultado não for vazio, "NA" nem a própria URL, job.options['folder'] é trocado por uma subpasta com esse nome (sanitizada por sanitize_filename()) antes do template de saída ser calculado — por isso essa checagem precisa rodar **antes** de output_template_fn(job) em _run_job().

Para o status "Título - Artista (X de Y)" em tempo real, build_command() sempre adiciona dois --print, usando os pontos de disparo reais do yt-dlp before_dl (antes de cada vídeo começar a baixar) e after_move (depois do arquivo final estar pronto), com um prefixo próprio (BDL@@@/ADL@@@) e campos separados por @@@ para serem fáceis de reconhecer e quebrar em NOW_PLAYING_RE:

```
--print before_dl:BDL@@@%(title)s@@@%(artist|)s@@@%(playlist_index|)s@@@%(playlist_coun
t|)s
--print after_move:ADL@@@%(title)s@@@%(artist|)s@@@%(playlist_index|)s@@@%(playlist_cou
nt|)s
```

%(artist|)s usa só o metadado real de artista (presente sobretudo em links do YouTube Music), com string vazia como padrão quando não existe — **de propósito sem cair para uploader (nome do canal)**. Uma primeira versão usava %(artist,uploader,creator|)s como fallback, mas isso fazia o nome de canais que só

reúpam remixes/covers (ex.: 'Canal X') virar um prefixo errado no nome do arquivo e nos metadados — 'Canal X - Nome da Música' em vez de só 'Nome da Música'. As duas flags foram validadas rodando o yt-dlp baixado de verdade contra uma URL inválida: o erro retornado foi de rede (DNS), não de parsing de argumento — confirmando que a sintaxe é aceita.

Quando uma dessas linhas chega no loop de leitura do stdout em _run_job(), um evento now_playing é emitido (em vez do evento genérico log) e tratado em App._handle_queue_event(), que monta o texto localizado e grava em job.current_label — lido depois tanto pelo status da aba quanto por QueueRow.refresh().

Busca automática de título/artista/duração

Ao colar exatamente um link numa aba (áudio ou vídeo), DownloadTabBase agenda uma chamada a downloader.probe_media_info(url) 900 ms depois da última tecla apertada (_on_link_changed cancela e reagenda o self.after() a cada tecla — debounce simples), rodando numa thread separada para nunca travar a UI. A função roda um comando rápido, sem baixar nada:

```
yt-dlp --print "%(title|)s@@@%(artist|)s@@@%(duration|0)s"
       --skip-download --playlist-items 1 URL
```

O resultado volta pela mesma fila thread-safe já usada pelos downloads (self._probe_results, um queue.Queue por aba, lido a cada 200 ms por _poll_probe_results). Um número de sequência (self._probe_seq) descarta resultados de uma busca antiga caso o usuário troque o link antes dela terminar. O resultado atualiza a duração/slider em DownloadTabBase._apply_probe_result() e, na aba de áudio, também título/artista (só se esses campos ainda estiverem vazios — AudioTab sobrescreve o método e chama super() primeiro). Com mais de um link colado, _start_probe() retorna sem fazer nada: título/artista/recorte valem para todos os itens de uma leva em _make_jobs(), então preenchê-los com dados só do primeiro link aplicaria esses dados errado nos outros.

Nome do arquivo final: duas regras separadas

App.build_output_template(job), em main.py, é onde o template -o passado ao yt-dlp é decidido — e áudio e vídeo seguem regras propositalmente diferentes, sem nenhum código compartilhado entre elas:

```
if job.mode == "audio":
    # SÓ o título, nunca o artista, mesmo se job.artist existir
    return f"{pasta}/{job.title ou '%(title)s'}.%(ext)s"
# vídeo: título + canal, resolvido pelo próprio yt-dlp
return f"{pasta}/%(title)s - %(channel,uploader)s.%(ext)s"
```

Essa separação existe porque o campo artist do YouTube (usado tanto pelo probe_media_info() quanto pelos metadados embutidos) nem sempre é confiável — em vídeos sem uma ficha técnica musical de verdade, pode vir preenchido com algo tão inútil pra nome de arquivo quanto o nome do canal que subiu o remix/cover. Em vez de tentar adivinhar quando esse dado é confiável, o nome do arquivo de áudio simplesmente **nunca** usa job.artist — só job.title (preenchido pelo probe ou digitado à mão) ou, na ausência dele, o %(title)s resolvido pelo próprio yt-dlp. O campo Artista continua enviado para --postprocessor-args (metadados/ID3 do arquivo, seção 5), então mudar essa política de nomes não afeta os metadados internos.

Para vídeo, %(channel,uploader)s é resolvido inteiramente pelo próprio yt-dlp no momento do download (sintaxe de campos alternativos, igual à usada em probe_media_info()) — não depende de nenhuma busca prévia do lado do app. Em playlists, _resolve_playlist_folder() zera job.title/job.artist (preenchidos pela busca com a 1ª faixa) antes de montar o template e o comando — sem isso todas as faixas iam para

o mesmo nome de arquivo e recebiam o mesmo -metadata title=, sobrescrevendo-se.

Thumbnail: regras de JPG e recorte 1:1

A decisão de quando salvar/converter/recortar uma thumbnail está toda concentrada em poucas linhas no início de build_command(). Não existe uma configuração separada pra isso — em áudio, o recorte quadrado é sempre automático, tanto pra capa embutida quanto pra salva em disco (uma versão anterior tinha um toggle 'Capa quadrada 1:1' em Configurações, removido por ser redundante: não há cenário onde faz sentido querer uma capa de música NÃO quadrada):

```
save_thumb_file = ... # 'Guardar thumbnail' OU (playlist + salvar capa)
want_thumbnail = save_thumb_file or embed_metadata_thumb
want_square_crop = job.mode == "audio" and want_thumbnail
want_jpg_convert = save_thumb_file or want_square_crop
```

O recorte 1:1 é sempre condicionado a job.mode == "audio" — uma thumbnail de vídeo nunca é recortada (fica no tamanho/proporção original), mesmo com 'Guardar thumbnail' marcado. --embed-thumbnail continua controlado só por 'Incorporar metadados e capa' (que já vem ligado por padrão); marcar apenas 'Guardar thumbnail' salva/recorta o arquivo em disco sem necessariamente embutir a capa no áudio.

6. O mecanismo de recorte de capa (--ppa e escaping)

O recorte 1:1 automático de capas de música (seção anterior) usa o mecanismo --ppa NOME:ARGS do yt-dlp para injetar argumentos extras no FFmpeg dentro do passo de conversão de thumbnail (postprocessor ThumbnailsConvertor), aplicando um filtro de recorte central:

```
crop='if(gt(ih,iw),iw,ih)':'if(gt(iw,ih),ih,iw)'
```

O desafio é que o yt-dlp faz seu próprio shlex.split() no valor de --ppa antes de repassar ao FFmpeg — e esse split remove aspas simples soltas, que são justamente o que protege as vírgulas da expressão contra serem lidas como separador de filtro pelo FFmpeg. A solução (em SQUARE_CROP_PPA, downloader.py) é envolver cada aspa simples literal em aspas duplas (""), um truque clássico de quoting estilo shell POSIX que sobrevive ao shlex.split:

```
_Q = the 3-character string:  "  '  "
SQUARE_CROP_PPA = (
    "ThumbnailsConvertor+FFmpeg_o:-c:v mjpeg -vf "
    f"crop={_Q}if(gt(ih,iw),iw,ih){_Q}:{_Q}if(gt(iw,ih),ih,iw){_Q}"
)
```

Isso foi validado isoladamente rodando shlex.split() em Python sobre o valor final e conferindo que o token resultante é exatamente crop='if(gt(ih,iw),iw,ih)':'if(gt(iw,ih),ih,iw)', com as aspas simples preservadas. **Se essa string precisar ser alterada, revalide sempre com um teste de shlex antes de testar com o yt-dlp de verdade** — é fácil quebrar o escaping sem perceber.

Quando o job é uma playlist detectada (ver seção 5) e a opção 'Salvar capa do álbum/playlist na pasta' está ativa (ligada por padrão), promote_folder_cover() roda depois que o job termina com sucesso: procura a imagem de thumbnail mais antiga na subpasta da playlist (heurística: geralmente é a primeira faixa) e copia/converte para playlistcover.jpg, usando o próprio FFmpeg já empacotado no app quando o formato original não é jpg.

Capa de playlist, verificação e aba Converter

Capa de playlist: com a playlist detectada e 'Guardar thumbnail' (ou 'Salvar capa na pasta') ligado, build_command() adiciona -o "pl_thumbnail:PASTA/playlist_thumb.%(ext)s" (tipo de template documentado do yt-dlp) para a capa da própria playlist ter nome fixo. Ao terminar, promote_folder_cover() prefere esse arquivo (cai para a imagem mais antiga se a playlist não tiver capa), converte para playlistcover.jpg recortando 1:1 em áudio, e cleanup_stray_thumbnails() apaga as capas das faixas.

Verificação: downloader.check_playlist(url, emit) é a porta do script PowerShell: --flat-playlist --print "%(id)s\t%(title)s" mapeia posição/título, depois -i -s --print OK:%(id)s simula a playlist inteira e cada linha ERROR: [x] ID: motivo (regex PL_ERR_RE) vira uma linha no Log com as vizinhas. Roda numa thread; emit é a mesma fila thread-safe do Log.

Aba Converter (converter_tab.py): build_ffmpeg_cmd() monta o comando a partir de duas tabelas (AUDIO: bitrate por qualidade; VIDEO: CRF por qualidade) e limita a resolução com scale=-2:'min(ih,H)' (nunca amplia). O progresso vem de -progress pipe:1: a duração sai da linha Duration: e o andamento de out_time_us=, ambos no stdout com stderr mesclado. Para novo formato, basta uma linha em AUDIO/VIDEO. Compressor: com target_bytes, probe_duration() (ffprobe) lê a duração e o bitrate vira bytes×8/duração×0,97 (3% de folga pro contêiner); em vídeo, desconta o -b:a da tabela e usa -b:v/-maxrate/-bufsize numa só passagem (o -b:v no fim sobrescreve o -b:v 0 do VP9, pois no ffmpeg vale a última opção). Alvo impossível (<100 kbps de vídeo) ou FLAC/WAV geram ValueError, mostrado no Log. Se precisar de tamanho exato, o próximo passo é 2-pass (-pass 1/2). src/selfcheck.py testa sem rede o comando do conversor, o template da capa e o parser da verificação.

7. Gerenciamento de dependências

yt-dlp e FFmpeg **não** são embutidos no .exe — são baixados na primeira execução e guardados numa pasta data/bin/ criada bem ao lado do próprio executável (modo portátil: fácil de achar, copiar ou apagar — nada escondido em AppData). Isso mantém o instalador pequeno e garante que o usuário sempre tenha uma versão razoavelmente recente do yt-dlp (que precisa de atualizações frequentes para acompanhar mudanças nos sites).

settings_store.get_app_data_dir() resolve essa pasta a partir de sys.executable quando rodando como .exe empacotado (nunca de __file__, que apontaria para a pasta temporária que o PyInstaller usa em modo --onefile) — com um fallback automático para %APPDATA%/YT-DLP Studio caso a pasta ao lado do .exe não possa ser criada/escrita (ex.: instalado em Arquivos de Programas sem permissão de administrador).

Binário	Fonte	Tamanho aprox.
yt-dlp.exe	github.com/yt-dlp/yt-dlp/releases/latest	~15 MB
ffmpeg.exe / ffprobe.exe	gyan.dev/ffmpeg/builds (pacote essentials)	~80 MB

O botão 'Atualizar yt-dlp' simplesmente re-executa install_ytdlp(), sobrescrevendo o binário local pela versão mais recente do GitHub. Se algum desses domínios mudar no futuro, basta atualizar as constantes YTDLP_RELEASE / FFMPEG_WIN_ESSENTIALS no topo de dependencies.py.

8. Fila de downloads e threading

Regra de ouro do projeto: **nenhuma thread de fundo toca em um widget diretamente.** Toda comunicação passa por uma queue.Queue (QueueManager.events), e só a thread principal (a que roda o

mainloop() do Tkinter) lê essa fila, via App._poll_queue_events reagendado a cada 150 ms com self.after(). Isso evita os travamentos/erros clássicos de Tkinter quando uma thread mexe na UI diretamente.

QueueManager.start() sobe uma *thread* daemon (_run_loop) que percorre self.jobs em ordem, chamando _run_job() para cada um. Cada job roda o yt-dlp via subprocess.Popen, lê a saída linha a linha, extrai a porcentagem com uma regex (PROGRESS_RE) e emite eventos job_update/log a cada linha nova.

'Cancelar Download Atual' seta um threading.Event (_stop_flag) e termina o processo do yt-dlp em andamento; isso cancela **só o item atual** — a flag é limpa logo em seguida, então os próximos itens da fila continuam normalmente.

9. Persistência de configurações

Um único arquivo JSON guarda tudo: pastas padrão, todas as opções avançadas, e também os últimos valores escolhidos em cada dropdown das abas (chaves last_*), para que o programa reabra do jeito que o usuário deixou. SettingsStore.save() é chamado explicitamente após qualquer mudança relevante (não há salvamento automático 'em tempo real' campo a campo).

10. Internacionalização (i18n)

Toda string visível vive em i18n.STRINGS['pt'] e ['en'], com a mesma chave nos dois dicionários. Cada tela expõe um método retranslate() que reconfigura o texto de cada widget guardado em self._i18n_labels (um dicionário widget→chave) — chamado sempre que o idioma muda (botão PT/EN no topo).

Para adicionar um novo idioma

1. Copie o bloco 'en': {...} em i18n.py, traduza todos os valores, e dê a ele uma nova chave (ex.: 'es').
2. No botão de idioma (main.py → toggle_language), troque a lógica binária 'pt'/'en' por uma lista de idiomas suportados.

11. Guia prático: como adicionar/alterar coisas comuns

Adicionar um novo formato de áudio

1. Em tabs.py → AudioTab.AUDIO_FORMATS_TECH, adicione o nome que vai aparecer no menu (ex.: 'WMA').
2. Em downloader.py → AUDIO_FORMAT_VALUE, mapeie esse nome para o valor que o yt-dlp espera em --audio-format (ex.: 'WMA': 'wma').

Adicionar uma nova resolução de vídeo

1. Em tabs.py → VideoTab.RESOLUTIONS_TECH, adicione o rótulo (ex.: '8640p (16K)').
2. Em downloader.py → RES_HEIGHT, mapeie para a altura em pixels (ex.: '8640p (16K)': 8640).

Adicionar uma nova opção de Configurações

1. Adicione o valor padrão em settings_store.py → DEFAULTS.

2. Adicione as strings PT/EN do rótulo em i18n.py.
3. Adicione o campo em settings_dialog.py (uma linha em self.entries[...] ou self.checks[...], dentro da seção apropriada — os métodos _load_values/_save já percorrem esses dicionários automaticamente, não precisa mexer neles).
4. Use a nova chave dentro de build_command() em downloader.py (s.get('sua_chave')).

Alterar o texto da Ajuda dentro do app

Edite a lista HELP_CONTENT['pt'] / ['en'] em help_dialog.py — cada item é uma dupla (título da seção, texto).

12. Build e empacotamento (gerando o .exe)

O projeto é distribuído como código-fonte Python + build.bat, não como um .exe pré-compilado — isso porque o PyInstaller empacota para a plataforma onde é executado; gerar um .exe Windows exige rodar o build em uma máquina Windows (ou um ambiente de cross-compilation, que não é usado aqui por simplicidade e confiabilidade).

O que build.bat faz, passo a passo:

1. Cria um ambiente virtual Python (.venv).
2. Instala as dependências de requirements.txt (customtkinter, pillow, pyinstaller).
3. Roda o PyInstaller em modo --onefile --windowed, embutindo a pasta assets/ (ícone e logo) dentro do executável.
4. O resultado final fica em dist/YT-DLP Studio.exe — um único arquivo, sem necessidade de Python instalado na máquina de destino.

Em tempo de execução, o app localiza seus próprios assets com resource_path() (main.py), que verifica sys._MEIPASS — a pasta temporária que o PyInstaller cria ao rodar um .exe --onefile. Fora do .exe (rodando via python src/main.py direto), ele cai de volta para a pasta assets/ ao lado do projeto.

13. Solução de problemas para desenvolvedores

Sintoma	Causa provável / onde olhar
App abre mas trava ao clicar em algo	Alguma thread de fundo está chamando um método de widget diretamente — revise se o código novo está passando pelo padrão queue.Queue + self.after() em vez de mexer na UI direto.
Download falha sempre com o mesmo erro	Rode o comando exato copiado do painel de Log direto no terminal — o log já imprime a linha completa '$ yt-dlp ...' antes de cada execução.
Opção nova na UI não tem efeito no download	Confira se a chave usada em tabs.py/settings_dialog.py é exatamente a mesma lida em downloader.py → build_command() (erros de digitação em nomes de chave falham silenciosamente, sem exceção).
PyInstaller gera .exe mas ele não abre	Rode o build sem --windowed uma vez para ver o traceback no console; geralmente é um import faltando em --hidden-import ou um asset não incluso em --add-data.

14. Versão Tauri + TypeScript (pasta tauri/)

A pasta tauri/ contém um porte completo do app, com as mesmas quatro abas e as mesmas regras de negócio. A interface é HTML/CSS/TypeScript puros (sem framework) rodando no WebView2 do próprio Windows, e uma casca pequena em Rust cuida do que a página não pode fazer. O resultado é um único .exe de cerca de 4 MB, com yt-dlp, FFmpeg e configurações numa pasta data/ ao lado. É independente da versão Python.

Arquivo	Equivalente Python / responsabilidade
src/core.ts	downloader.build_command + converter_tab.build_ffmpeg_cmd + check_playlist. Funções puras (testadas no selfcheck).
src/backend.ts	settings_store.py, dependencies.py, QueueManager e o worker do conversor. Expõe o objeto api para a interface.
src/renderer.ts	tabs.py, queue_tab.py, settings_dialog.py e help_dialog.py.
src/i18n.ts	GERADO por tools/gen_i18n.py a partir de src/i18n.py e src/help_dialog.py. Não editar à mão.
ui/index.html, ui/style.css	Estrutura da tela e a paleta (variáveis CSS, tema claro/escuro).
src-tauri/src/main.rs	Casca nativa, sem lógica do app: iniciar/matar processos, download HTTP, arquivos, diálogos e a janela.

```
cd tauri
npm install        # uma vez
npm start          # compila e abre o app
npm run check      # selfcheck offline, tem que imprimir "selfcheck OK"
npm run dist       # gera release\YT-DLP Studio.exe
```

Compilar exige Node.js, Rust e o Visual Studio C++ Build Tools (a primeira compilação leva uns 15 minutos). Rodar o .exe não exige nada além do WebView2, que já vem no Windows 10/11 atualizado.

Diferenças em relação à versão Python:

- No lugar de queue.Queue + self.after(): as threads do Rust só emitem eventos (proc, exit, dl) e a interface fala com o backend.ts por api.call(nome, ...) e api.on().
- O yt-dlp é chamado com --encoding utf-8 (acentos corretos pelo pipe) e --progress (o --print implica --quiet e esconderia as linhas de porcentagem).
- Cancelar usa taskkill /T /F, que encerra também o ffmpeg filho.
- As seleções das abas ficam em settings.json → ui por valor técnico (ex.: audio.format = mp3), não pelo texto traduzido.
- O zip do FFmpeg é extraído com o tar.exe do Windows.
- Faixas com erro (só nesta versão): o --print after_move traz também %(id)s e %(filepath)s, e as linhas ERROR: [extrator] ID: motivo são coletadas. No fim da playlist, locateFailed() usa --flat-playlist para achar a posição de cada faixa; replaceFailed() baixa a substituta com --no-playlist na mesma pasta e grava as datas de criação/modificação com core.between() (meio entre as vizinhas) pelo comando Rust set_times. pause_failed vale each (a fila espera em resumeQueue depois da playlist), end (evento review no fim da fila) ou off.

- Pastas: o yt-dlp recebe -P pasta_destino e -P temp:data\tmp com -o relativo, então .part, .webm e a capa a embutir nunca aparecem na pasta de destino. Numa playlist, a capa da playlist vem de um comando à parte (core.playlistCoverArgs(): --flat-playlist --playlist-items 0 --write-thumbnail), sem tocar nas faixas; capas por faixa só com save_track_covers (e -o "pl_thumbnail:" vazio, para não gravar a capa da playlist com o nome dela).
- Nome na fila: a busca automática usa --flat-playlist e devolve %(playlist_title)s; itens adicionados sem nome são nomeados em segundo plano, um por vez (nameLater()). job.name só é exibido; nunca entra no nome do arquivo.
- Cookies: core.cookieArgs() adiciona --cookies-from-browser edge|chrome em todas as chamadas do yt-dlp quando use_cookies está ligado. Mensagens conhecidas (DPAPI, banco de cookies travado, *not a bot*) ganham uma explicação no Log (HINTS).